signal-lab — Complete Code Walkthrough

Every file, line by line, explained for someone who knows Python but is new to backend, databases, finance, and LLMs.

Read Part 0 first — it defines the vocabulary (`conn`, `cursor`, ticker, API, JSON, Sharpe, etc.) once, so the file-by-file sections don't have to keep re-explaining. Then the files are explained in a *learning* order (simplest concepts first), which is slightly different from the *run* order.

Part 0 — Concepts Glossary

A. How programs talk to the internet

API (Application Programming Interface). A way for your program to ask another program for data or services over the internet. Think of a restaurant: you (your script) don't walk into the kitchen; you give the waiter an order from a menu (the API's list of allowed requests), and the kitchen sends back a dish (the data). Alpha Vantage's API is the "kitchen" that has stock data; your script is the customer.

HTTP request / `GET`. The mechanism for making that order. A `GET` request means "please *give* me some data" (as opposed to `POST`, which means "here's some data, please *store/process* it"). `requests.get(url, params=...)` sends a GET request.

Endpoint + parameters. The endpoint is *which* menu item you're ordering (a URL like `https://www.alphavantage.co/query`). Parameters are the customizations (`symbol=AAPL`, `function=TIME_SERIES_DAILY`) — like "medium rare, no onions." They get attached to the URL as `?symbol=AAPL&function=...`.

JSON (JavaScript Object Notation). The text format the data comes back in. It looks almost exactly like a Python dictionary — curly braces, `"key": value` pairs, lists in square brackets. When you call `resp.json()`, Python converts that text into real Python dicts and lists you can index into.

CORS (you asked about this — note: it does NOT appear in any of these files). CORS = Cross-Origin Resource Sharing. It's a *browser* security rule that controls whether a web page loaded from site A is allowed to call an API on site B. It only matters when a website's JavaScript calls your API. Your scripts here are plain Python running on your own machine, not a browser, so CORS is irrelevant *for now*. It will come back when you build the Streamlit/web dashboard that talks to a backend — I'm defining it so the term isn't a mystery, not because it's used yet.

B. Databases and SQLite

Database. An organized store of data on disk that you query with a language called SQL. Think of it as a very disciplined spreadsheet program you talk to in text commands.

SQLite. The specific database you're using. Its special trait: the *entire database is one file* (`data/market.db`) and there's no separate server to install or start — Python's built-in `sqlite3` module reads and writes that file directly. That's why it's perfect for learning.

Table / row / column. A table is one grid of data (e.g., `daily_prices`). A column is a field (`close`, `volume`). A row is one record (one day's prices for one ticker).

`conn` (connection). The open link between your Python program and the database file. `conn = sqlite3.connect("data/market.db")` opens the file. Analogy: `conn` is the *phone line* you've opened to the database.

`cursor` (`cur`). An object that actually runs SQL commands over that connection and holds the results you get back. Analogy: if `conn` is the phone line, the `cursor` is the *person on the call* reading out your commands and writing down the answers. You can often run commands directly on `conn` (a shortcut SQLite allows), which is why some files use `conn.execute(...)` and others create `cur = conn.cursor()` first — both work.

`commit`. Databases hold your changes in a temporary "pending" state until you confirm them. `conn.commit()` = "save it for real." If you never commit, your inserts vanish when the program ends. Analogy: typing a document vs. hitting Save.

`close`. `conn.close()` hangs up the phone line and frees the file. Good hygiene.

SQL (Structured Query Language). The text language for talking to databases. Key verbs you'll see:

- `CREATE TABLE ...` — make a new table with named columns and types.
- `INSERT INTO ... VALUES ...` — add rows.
- `SELECT ... FROM ... WHERE ...` — read rows that match a condition.
- `JOIN` — combine two tables on a shared column.
- `GROUP BY` — collapse many rows into one per group (e.g., one per ticker) so you can count or average.
- `DROP TABLE` — delete a whole table.

Column types. `TEXT` (strings), `REAL` (decimal numbers, e.g. prices), `INTEGER` (whole numbers, e.g. volume). SQLite is relaxed about these but it's good practice to declare them.

Primary key. A column (or combination) that uniquely identifies each row, and that the database *enforces* as unique. `PRIMARY KEY (symbol, date)` means "there can only be one row per (symbol, date) pair." This is what stops duplicates.

Foreign key. A column that points to another table's primary key, documenting a relationship. `FOREIGN KEY (url) REFERENCES news_articles(url)` says "every url in this table should correspond to an article in `news_articles`."

`INSERT OR REPLACE`. Normally, inserting a row whose primary key already exists throws an error. `INSERT OR REPLACE` instead *overwrites* the existing row. This makes your scripts **idempotent** — a fancy word meaning "running it twice gives the same result as running it once, no duplicates piling up."

Parameterized query / the `?` placeholders. Look at:

```python
cur.execute("INSERT INTO daily_prices (...) VALUES (?, ?, ?, ...)", rows)
```

The `?` marks are placeholders. You never paste values straight into the SQL string. Two reasons: (1) **safety** — pasting raw text lets malicious input rewrite your query (called SQL injection); the `?` approach passes values *separately* so they can never be interpreted as commands. (2) **correctness** — the database handles quoting and type conversion for you. `executemany(sql, rows)` runs that same insert once per item in the `rows` list.

C. pandas (working with tables in memory)

DataFrame (`df`). pandas' in-memory table — like a spreadsheet living in a Python variable. `pd.read_sql_query("SELECT ...", conn)` runs SQL and hands you the result as a DataFrame.

`df["column"]`. One column (a Series). `df[["a","b"]]` (double brackets) selects several columns as a smaller DataFrame.

`groupby`. Splits the table into groups (e.g., one group per ticker), lets you compute something per group, then recombines. Essential here because your table stacks 5 tickers on top of each other, and most calculations must stay *within* one ticker.

`.transform(...)`. Runs a per-group calculation and returns a result *aligned back to every original row*. Used for moving averages so each row gets its own ticker's average, not a mix.

`.pct_change()`. Percentage change from the previous row. On grouped data (`groupby("symbol")["close"].pct_change()`) it resets at each ticker boundary.

`.shift(1)`. Moves a column *down* by one row, so each row now sees "yesterday's" value. Critical for avoiding lookahead bias (Section E).

`.fillna(0)`. Replaces missing values (`NaN`) with 0. Missing values appear at the start of moving averages (not enough history yet) and on days with no news.

`.merge(...)`. The pandas version of a SQL JOIN — glue two DataFrames together on shared columns.

D. Finance vocabulary

Ticker / symbol. A short code for a company's stock. `AAPL` = Apple, `MSFT` = Microsoft, `GOOGL` = Alphabet/Google, `AMZN` = Amazon, `META` = Meta/Facebook. You're tracking those five.

OHLC + Volume. For each trading day the market records: **Open** (first traded price), **High** (highest), **Low** (lowest), **Close** (last price of the day — the one everyone quotes), and **Volume** (how many shares changed hands). Close is what your strategy uses.

Daily return. The percent change in close from one day to the next. If AAPL closes at 100 then 102, the daily return is +2% (0.02). This is the fundamental unit of "did I make or lose money today."

SMA (Simple Moving Average). The average close over the last N days, recomputed each day. A **5-day SMA** reacts fast; a **20-day SMA** is slower/smoother. When the fast SMA rises *above* the slow one, it's a common (crude) signal that the short-term trend is turning up. That's exactly the rule your `build_signal` uses.

Sentiment score. A number (here roughly −1 to +1) saying how positive or negative a news article is. **Relevance score** (0 to 1) says how *central* a given company is to that article. Multiplying them (`sentiment × relevance`) weights a strong opinion in an article that's really about Apple more than the same opinion in an article that just mentions Apple in passing.

Signal. A rule that outputs "be in the market" (1) or "stay out" (0) for each day.

Position. What you're actually *holding*: 1 = you own the stock that day, 0 = you're in cash. It's the signal shifted forward by a day (you can only act *after* you see the signal).

Backtest. Simulating your strategy on historical data to see how it *would* have done. Cheap way to sanity-check an idea before risking money.

E. The four ideas interviewers probe on a backtest

Lookahead bias. Accidentally using information you couldn't have known at the time. Example: deciding to buy *on* Monday using Monday's closing price — but you'd only know Monday's close *after* the day ends. The `shift(1)` fixes this by making today's trade depend on *yesterday's* signal.

Transaction costs. Every time you buy or sell, you pay a small fee/spread. `COST = 0.001` = 0.1% charged whenever your position changes. A strategy that looks great but flips every day often dies once costs are subtracted — which is exactly why you include them.

Sharpe ratio. Return earned *per unit of risk*. Formula here: `(mean daily return / std of daily returns) × √252`. Higher is better. The `√252` "annualizes" it (there are ~252 trading days in a year). A Sharpe around 1 is decent; above ~3 on only 100 days should make you *suspicious of a bug*, not proud.

Max drawdown. The worst peak-to-trough drop your account would have suffered — e.g., −20% means at some point you were down 20% from your previous high. Measures pain, not just average return.

Hit rate. Of the days you were actually trading, what fraction were profitable.

Equity curve. Your simulated account value over time, starting at 1.0. `(1 + returns).cumprod()` multiplies each day's growth factor together to trace it.

F. LLM vocabulary

LLM (Large Language Model). An AI that reads text and writes text (e.g., Qwen, Claude). Here it reads a headline + summary and writes a structured judgment.

Hugging Face `InferenceClient`. A way to call a hosted model over the internet without running it on your own computer — same client/API idea as Section A, just aimed at an AI model instead of a stock database.

System prompt vs user message. The **system prompt** sets the rules and role ("You are a financial news analyst; reply only in this JSON schema"). The **user message** is the specific input for this call (this article). The model reads both.

Temperature. A knob for randomness, 0 to ~1. Low (`0.2`) = focused, consistent, repeatable — what you want for classification. High = creative/varied.

`max_tokens`. A cap on how long the reply can be. Tokens ≈ word-pieces. It limits length and cost.

Structured output / JSON parsing. You *ask* the model to return only JSON so you can `json.loads()` it straight into a database row. Models sometimes disobey (add prose or code fences), so the code defensively finds the first `{` and last `}` and parses only that slice.

Part 1 — `fetch_prices.py` (start here — the simplest file)

Job: ask Alpha Vantage for the last ~100 daily prices of each ticker and store them in the `daily_prices` table.

```
python
```

```
import os
import sqlite3
import requests
from dotenv import load_dotenv
import time
```

- `os` — read environment variables (your secret API key) and make folders.
- `sqlite3` — Python's built-in database module.
- `requests` — makes the HTTP calls to the API.
- `dotenv` — loads variables from your `.env` file.
- `time` — lets you pause (`time.sleep`) between API calls.

python

```
load_dotenv()
API_KEY=os.getenv("ALPHAVANTAGE_API_KEY")
```

`load_dotenv()` reads your `.env` file and injects its lines into the environment. `os.getenv("ALPHAVANTAGE_API_KEY")` then pulls your key out. **Why not just paste the key in the code?** So you never accidentally commit a secret to GitHub. The `.env` file is git-ignored; the code only references the *name* of the variable.

python

```
TICKERS=["AAPL","MSFT","GOOGL","AMZN","META"]
DB_PATH="data/market.db"
BASE_URL="https://www.alphavantage.co/query"
```

Constants: the five companies, where the database file lives, and the API endpoint. Putting these at the top (in capitals, by convention) means you change them in one place.

python

```
def fetch_daily(symbol):
    params={
        "function":"TIME_SERIES_DAILY",
        "symbol":symbol,
        "outputsize":"compact",
        "apikey":API_KEY
    }
```

Builds the "order" for the API. `function` picks which data (daily time series). `symbol` is the company. `outputsize="compact"` asks for the last 100 days (vs `"full"` = 20+ years). `apikey` authenticates you.

```python
    resp=requests.get(BASE_URL, params=params, timeout=30)
    resp.raise_for_status()
    data=resp.json()
```

- `requests.get(...)` sends the request; `params` get appended to the URL as `?function=...&symbol=...`. `timeout=30` means "give up after 30 seconds" so a dead server can't hang your program forever.
- `resp.raise_for_status()` — if the server returned an HTTP error code (like 404 or 500), turn it into a Python exception instead of silently continuing.
- `resp.json()` converts the JSON text reply into a Python dict named `data`.

```python
    if "Time Series (Daily)" not in data:
        raise RuntimeError(f"unexpected response from api : {data}")
    return data["Time Series (Daily)"]
```

Alpha Vantage returns a **200 OK with no price data** when you hit a rate limit or make a bad request — the error hides *inside* the JSON. So you check that the expected key exists; if not, you raise an error showing what actually came back (often a "you've hit your daily limit" note). If it's there, you return that inner dictionary, which maps each date string to its OHLCV values.

```python
def save_to_sqlite(symbol,series):
    os.makedirs(os.path.dirname(DB_PATH), exist_ok=True)
    conn=sqlite3.connect(DB_PATH)
    cur=conn.cursor()
```

- `os.makedirs("data", exist_ok=True)` creates the `data/` folder if it doesn't exist (`exist_ok=True` = "don't error if it's already there").
- `sqlite3.connect(DB_PATH)` opens the phone line (`conn`) to the database file, creating the file if needed.
- `conn.cursor()` gets the "person on the call" (`cur`) who'll run commands.

```python
    cur.execute(
        """
        CREATE TABLE IF NOT EXISTS daily_prices(
        symbol text not null,
        date text not null,
        open real, high real, low real, close real,
        volume integer,
        primary key(symbol, date)
        )
        """
    )
```

Runs a `CREATE TABLE IF NOT EXISTS` — makes the table only if it isn't already there (so re-running is safe). `not null` means those columns must always have a value. `primary key(symbol, date)` enforces one row per company per day — this is what makes `INSERT OR REPLACE` able to overwrite instead of duplicate.

```python
    rows=[
        (symbol, date,
         float(values["1. open"]), float(values["2. high"]),
         float(values["3. low"]),  float(values["4. close"]),
         int(values["5. volume"]))
        for date,values in series.items()
    ]
```

A **list comprehension** that turns the API's nested dict into a flat list of tuples, one per day. `series.items()` gives `(date, values)` pairs; `values` is a dict like `{"1. open": "150.2", ...}`. The API sends numbers **as strings**, so `float(...)` and `int(...)` convert them to real numbers — otherwise your database would store `"150.2"` as text and math later would break.

```python
```

```
    cur.executemany(
    """
    INSERT OR REPLACE INTO daily_prices
    (symbol, date, open, high, low, close, volume)
    VALUES (?, ?, ?, ?, ?, ?, ?)
    """,
    rows
    )
```

`executemany` runs the insert once for every tuple in `rows`. The seven `?` line up with the seven columns; each tuple supplies the seven values safely (Section B, parameterized queries). `INSERT OR REPLACE` overwrites any row with a matching (symbol, date).

python

```
    conn.commit()
    conn.close()
    return len(rows)
```

Save the changes for real, hang up, and report how many rows were written.

python

```
def main():
    if not API_KEY:
        raise SystemExit("set ALPHAVANTAGE_API_KEY in .env file")
    for symbol in TICKERS:
        series=fetch_daily(symbol)
        count=save_to_sqlite(symbol, series)
        print(f"saved {count} rows for {symbol} to {DB_PATH}")
        time.sleep(15)
```

Guard clause: if the key didn't load, stop with a clear message. Then loop the five tickers: fetch, save, print. `time.sleep(15)` pauses 15 seconds between calls so you stay under Alpha Vantage's rate limit (free tier allows only a handful of calls per minute and 25/day).

python

```
if __name__=="__main__":
    main()
```

Standard Python idiom: "only run `main()` if this file was executed directly (`python fetch_prices.py`), not if it was imported by another file." Every script here ends this way.

Part 2 — `read_prices.py` (a scratch/exploration file, teaches pandas)

Job: none in the pipeline — it's a sandbox to *look* at your price data. Handy for learning.

```python
conn=sqlite3.connect("data/market.db")
df=pd.read_sql_query("SELECT * FROM daily_prices", conn)
```

Open the DB and pull the *entire* `daily_prices` table into a pandas DataFrame. `SELECT *` = "all columns."

```python
print(df.head())
print(df.info())
```

`.head()` shows the first 5 rows (a quick peek). `.info()` prints the columns, their types, and how many non-null values each has — your first stop when something looks off.

```python
df=df.sort_values(by=["symbol","date"])
print(df[["date","close"]].head(10))
```

Sort so each ticker's rows sit together in date order (essential before any time-based calc). Then print just date + close for the first 10 rows.

```python
df["return"]=df.groupby("symbol")["close"].pct_change()
```

Create a `return` column = daily percent change of close, **computed within each ticker** thanks to `groupby("symbol")`. (This is the corrected version — earlier you had it without the groupby, which computed nonsense returns across ticker boundaries.)

```python
df["range"]=df["high"]-df["low"]
df["m20"]=df.groupby("symbol")["close"].transform(lambda s: s.rolling(20).mean(
```

(Note: this file computes but never prints `range`/`m20` or saves anything — it's purely for you to inspect in a debugger or by adding prints. That's fine for a scratch file.)

Part 3 — `fetch_news.py` (two related tables)

Job: fetch news-with-sentiment for each ticker and store it across two tables.

Imports add `json` (to turn the `topics` list into a storable string) and `datetime`/`timezone` (to timestamp when you fetched). Same `load_dotenv` / `API_KEY` / constants pattern as before; `NEWS_LIMIT=50` caps articles per call.

```python
def fetch_news(symbol, retries=3):
    for i in range(retries):
        try:
            params = {"function":"NEWS_SENTIMENT","tickers":symbol,
                      "sort":"LATEST","limit":NEWS_LIMIT,"apikey":API_KEY}
            resp = requests.get(BASE_URL, params=params, timeout=30)
            resp.raise_for_status()
            data = resp.json()
            if "feed" not in data:
                raise RuntimeError(data)
            return data["feed"]
        except Exception as e:
            print(f"Retry {i+1}/{retries} failed for {symbol}: {e}")
            time.sleep(25)
    return []
```

Same request pattern as prices, but wrapped in a **retry loop**. `for i in range(retries)` tries up to 3 times. `try/except` means "attempt this; if *anything* goes wrong, jump to `except` instead of crashing." On failure it prints why, waits 25 seconds (often a rate-limit cooldown), and loops. If all 3 attempts fail it returns an empty list `[]` so the rest of the program can continue gracefully. The API's news field is called `"feed"` (a list of article dicts).

```python
def create_tables(conn):
    conn.executescript("""
        DROP TABLE IF EXISTS news_articles;
        DROP TABLE IF EXISTS news_ticker_sentiment;
        CREATE TABLE IF NOT EXISTS news_articles(
        url text primary key, title text, source text, source_domain text,
        time_published text, summary text,
        overall_sentiment_score real, overall_sentiment_label text,
        topics text, -- json string
        fetched_at text
        );
        CREATE TABLE IF NOT EXISTS news_ticker_sentiment(
        url text, ticker text, relevance_score real,
        ticker_sentiment_score real, ticker_sentiment_label text,
        primary key (url,ticker),
        foreign key (url) references news_articles(url)
        );
    """)
```

- `executescript` runs several SQL statements at once.
- **Why two tables?** One article can talk about several companies, each with its own sentiment. That's a **one-to-many relationship**. So `news_articles` holds one row per article (with the article's *overall* sentiment), and `news_ticker_sentiment` holds one row per (article, company) pair (the *per-company* sentiment). The `url` links them; the foreign key documents that link.
- `topics text, -- json string`: the fix from earlier — the comma must come **before** the `--` comment, or SQL treats the comment as eating the comma and merges two columns. `topics` is stored as a JSON string because it's a little list you won't need to query on.
- **Heads-up:** the `DROP TABLE IF EXISTS` lines mean every run *wipes and rebuilds* the news tables. That's fine on its own, but be aware: if you re-run this *after* `llm_extract.py`, you'll have LLM-analysis rows whose articles no longer exist (harmless orphans, but confusing). Re-fetch news only when you actually want fresh news.

```python
```

```
def save_news(conn,feed):
    fetched_at=datetime.now(timezone.utc).isoformat()
    article_rows,ticker_rows=[],[]
    for article in feed:
        url=article["url"]
        article_rows.append((... article.get("title"), ...
            float(article.get("overall_sentiment_score",0.0)),
            article.get("overall_sentiment_label"),
            json.dumps(article.get("topics",[])),
            fetched_at))
        for ticker in article.get("ticker_sentiment",[]):
            ticker_rows.append((url, ticker.get("ticker"),
                float(ticker.get("relevance_score",0.0)),
                float(ticker.get("ticker_sentiment_score",0.0)),
                ticker.get("ticker_sentiment_label")))
```

- `datetime.now(timezone.utc).isoformat()` = the current UTC time as a standard text string (e.g. `2026-06-13T09:30:00+00:00`), recording *when* you fetched.
- `.get("title")` vs `["title"]`: `.get()` returns `None` if the key is missing instead of crashing — safer for messy API data. `.get("overall_sentiment_score", 0.0)` supplies a default of 0.0 if absent.
- `json.dumps(...)` turns the Python `topics` list into a JSON string so it fits in one text column.
- The **inner loop** walks each article's `ticker_sentiment` list and builds one `ticker_rows` entry per company mentioned — this is what fills the second table. Again `float(...)` because the API sends these as strings.

python

```
    conn.executemany("INSERT OR REPLACE INTO news_articles (...) VALUES (?,?,?,
    conn.executemany("INSERT OR REPLACE INTO news_ticker_sentiment(...) VALUES
    return len(article_rows),len(ticker_rows)
```

Two bulk inserts — one per table — then report both counts. `ticker_rows` is usually *larger* than `article_rows` because each article tags multiple companies.

`main()` mirrors the prices file: check the key, create tables once, loop tickers (fetch → save → print), sleep 25s between calls, then commit and close. It prints per-ticker counts and a grand total.

Part 4 — `llm_extract.py` (the AI layer)

Job: for each AAPL/MSFT/GOOGL/AMZN/META-relevant article, ask an LLM to classify it and store the structured result in `news_llm_analysis`.

```python
from huggingface_hub import InferenceClient
MODEL="Qwen/Qwen2.5-7B-Instruct"
API_KEY=os.getenv("HUGGINGFACEHUB_API_KEY")
client=InferenceClient(token=API_KEY)
```

You're using Hugging Face's hosted models (free) instead of a paid API. `InferenceClient(token=...)` is your authenticated connection to the model — same client idea as `requests`, aimed at an AI. `MODEL` names which model to use (Qwen 7B, an instruction-following chat model).

```python
SYSTEM_PROMPT = """You are a financial news analyst. ... Respond with ONLY a va
```

The **system prompt** pins the model's role *and* forces a strict output contract: only JSON, exact keys, a fixed list of allowed `event_type` values, a one-sentence `rationale`, etc. The stricter this is, the more reliably you can parse the reply. This prompt *is* the "engineering" in AI engineering — it's the interface between an unpredictable model and your rigid database.

```python
def get_unprocessed_articles(conn):
    return conn.execute("""
        SELECT a.url, a.title, a.summary, t.ticker
        FROM news_articles a
        JOIN news_ticker_sentiment t ON a.url = t.url
        WHERE t.ticker IN('AAPL','MSFT','GOOGL','AMZN','META')
        AND NOT EXISTS (
            SELECT 1 FROM news_llm_analysis l
            WHERE l.url = a.url AND l.ticker = t.ticker )
        LIMIT 50
    """).fetchall()
```

- `JOIN ... ON a.url = t.url` stitches each article to its per-ticker rows, so you get title/summary *and* which ticker this row is about.

- `a` and `t` are **table aliases** (short nicknames) so you can write `a.url` instead of `news_articles.url`.
- `WHERE t.ticker IN (...)` keeps only your five companies (an article about AAPL often also tags NVDA, TSLA, etc.; you don't want to spend LLM calls on those).
- `AND NOT EXISTS (SELECT 1 FROM news_llm_analysis ...)` = "only rows we haven't analyzed yet." `SELECT 1` is a common idiom meaning "I don't care about the value, I just want to know if a matching row exists." This is what makes re-running cheap — already-done articles are skipped.
- `LIMIT 50` caps each run at 50 articles.
- `.fetchall()` pulls all matching rows back as a list of tuples.

python

```
def analyze(title, summary, ticker):
    messages=[
        {"role":"system","content":SYSTEM_PROMPT},
        {"role":"user","content":f"Company ticker: {ticker}\nHeadline: {title}\
    ]
    response=client.chat.completions.create(model=MODEL, messages=messages,
                                            max_tokens=250, temperature=0.2)
    text=response.choices[0].message.content
```

- `messages` is the standard chat format: a system message (rules) + a user message (this article's data). The f-string injects the actual title/summary/ticker.
- `client.chat.completions.create(...)` sends it to the model. `max_tokens=250` caps reply length; `temperature=0.2` keeps it consistent (Section F).
- `response.choices[0].message.content` digs the actual text reply out of the response object (`choices[0]` = the first/only completion).

python

```
    try:
        start=text.find("{")
        end=text.rfind("}")+1
        return json.loads(text[start:end])
    except Exception:
        return {"event_type":"other","theme":"unknown",
                "directional_lean":"neutral","confidence":0.0,"rationale":"pars
```

Defensive parsing. `text.find("{")` finds the first `{`; `text.rfind("}")` finds the *last* `}`; slicing between them extracts just the JSON even if the model wrapped it in chatter or ``` fences. `json.loads(...)` turns that JSON text into a Python dict. If *anything* fails (bad JSON, no braces), the `except` returns a safe fallback dict tagged `"parse error"` so one bad reply never crashes the whole run. **Tip:** count how many rows end up with `rationale='parse error'` — a high count means the model is misbehaving and the prompt needs tightening.

python

```
def create_table(conn):
    conn.execute("""CREATE TABLE IF NOT EXISTS news_llm_analysis (
        url text, ticker text, model text, event_type text, theme text,
        directional_lean text, confidence real, rationale text, analyzed_at tex
        PRIMARY KEY (url, ticker) )""")
```

Creates the results table (no `DROP` here — good, that's the fix that stopped it wiping itself every run). Primary key `(url, ticker)` = one analysis per article-company pair, which is exactly what `INSERT OR REPLACE` needs and what the `NOT EXISTS` check relies on. Storing `model` lets you later compare results across different models.

python

```
def main():
    conn=sqlite3.connect(DB_PATH)
    create_table(conn)
    rows=get_unprocessed_articles(conn)
    for url,title,summary,ticker in rows:
        result=analyze(title or "", summary or "", ticker or "")
        conn.execute("""INSERT OR REPLACE INTO news_llm_analysis (...)
                        VALUES (?,?,?,?,?,?,?,?,?)""",
            (url, ticker, MODEL, result.get("event_type"), result.get("theme"),
             result.get("directional_lean"), float(result.get("confidence")),
             result.get("rationale"), datetime.now(timezone.utc).isoformat()))
        time.sleep(5)
    conn.commit(); conn.close()
    print("LLM Analysis complete.")
```

Open DB, ensure the table exists, get the to-do list, then for each article: call the model, insert the structured result (with a timestamp), and `time.sleep(5)` to be polite to the free model endpoint. `title or ""` means "use title, but if it's `None`, use an empty string" — avoids passing `None` into the prompt.

⚠️ **One latent bug:** `float(result.get("confidence"))`. If the model returns valid JSON that happens to *omit* the `confidence` key, `.get("confidence")` returns `None`, and `float(None)` **crashes**. Change it to `float(result.get("confidence", 0.0))` (supply a default) so a missing key can't kill the run.

Part 5 — `generate_signals.py` (a tiny standalone report)

Job: a quick ranking of your five tickers by news sentiment. It's a standalone diagnostic, not part of the build → backtest chain.

python

```
query="""SELECT ticker,
    avg(ticker_sentiment_score*relevance_score) as weighted_sentiment,
    count(*) as mentions
FROM news_ticker_sentiment
WHERE ticker IN ('AAPL','MSFT','GOOGL','AMZN','META')
group by ticker order by mentions desc
"""
df=pd.read_sql_query(query, conn)
print("sentiment signal ranking\n",df)
```

- `avg(ticker_sentiment_score*relevance_score)` computes, per ticker, the *average relevance-weighted sentiment* — Section D's weighting idea, done in SQL. `avg(...)` and `count(*)` are **aggregate functions**: they collapse many rows into one number *per group*.
- `count(*) as mentions` counts how many articles mention each ticker.
- `GROUP BY ticker` is what makes those aggregates compute *per ticker* (one output row each). `ORDER BY mentions DESC` sorts most-mentioned first. This is the corrected version — earlier the clause order was wrong (`WHERE` before `FROM`) and string concatenation had a missing space; the triple-quoted string fixes both.

Part 6 — `build_features.py` (join prices + sentiment into one tidy table)

Job: produce the single `features` table your backtest consumes — one row per (symbol, date) with price features and a daily sentiment score merged in.

python

```python
def load_prices(conn):
    df=pd.read_sql_query("SELECT symbol,date,open,high,low,close,volume FROM da
    df["date"]=pd.to_datetime(df["date"])
    return df.sort_values(by=["symbol","date"]).reset_index(drop=True)
```

Load prices; `pd.to_datetime` converts the `date` *strings* into real date objects (so sorting and merging behave correctly). Sort by ticker then date. `reset_index(drop=True)` renumbers the rows 0,1,2,… after sorting and throws away the old numbering (`drop=True`) — just tidiness.

python

```python
def add_price_features(df):
    g=df.groupby(by=["symbol"])
    df["daily_return"]=g["close"].pct_change()
    df["sma_fast"]=g["close"].transform(lambda s: s.rolling(SMA_FAST).mean())
    df["sma_slow"]=g["close"].transform(lambda s: s.rolling(SMA_SLOW).mean())
    return df
```

`g` groups by ticker so every calc stays within one company. `daily_return` = per-ticker daily % change. `sma_fast`/`sma_slow` = 5-day and 20-day moving averages (Section D), aligned back to each row via `.transform`. `SMA_FAST`/`SMA_SLOW` are the constants defined up top.

python

```python
def load_daily_sentiment(conn):
    placeholders=",".join(f"'{t}'" for t in TICKERS)
    news=pd.read_sql_query(f"""
        SELECT a.time_published, t.ticker, t.ticker_sentiment_score, t.relevanc
        FROM news_articles a
        JOIN news_ticker_sentiment t ON a.url=t.url
        WHERE t.ticker IN ({placeholders})
    """, conn)
```

- `placeholders` builds the string `'AAPL','MSFT','GOOGL','AMZN','META'` by joining each ticker wrapped in quotes. It's then dropped into the `IN (...)` clause. (This works, though the cleaner/safer habit is parameter binding; fine here since `TICKERS` is your own trusted list.)
- The JOIN again marries article-level info (`time_published`) to per-ticker sentiment.

python

```
    if news.empty:
        return pd.DataFrame(columns=["time_published","ticker","ticker_sentimen
```

If there's no news at all, return an empty DataFrame so the caller doesn't crash. *(Minor note: the columns it returns here don't match the `symbol/date/sentiment` shape the merge later expects — harmless as long as news isn't empty, but worth tidying if you ever hit a truly newsless run.)*

python

```
    news["date"]=pd.to_datetime(news["time_published"].str[:8],format="%Y%m%d")
    news["weighted"]=news["ticker_sentiment_score"]*news["relevance_score"]
    daily=news.groupby(by=["ticker","date"])["weighted"].mean().reset_index() \
            .rename(columns={"ticker":"symbol","weighted":"sentiment"})
    return daily
```

- The API's `time_published` looks like `20250115T143000`. `.str[:8]` grabs the first 8 characters (`20250115`) = the calendar date; `format="%Y%m%d"` tells pandas how to read it. You reduce timestamps to *dates* because prices are daily.
- `weighted` = sentiment × relevance (Section D).
- `groupby(["ticker","date"])["weighted"].mean()` collapses possibly-many articles per ticker per day into **one average sentiment** for that ticker-day. `.reset_index()` turns the grouped result back into a normal table. `.rename(...)` renames columns to `symbol`/`sentiment` so they line up with the price table for merging.

python

```
def main():
    conn=sqlite3.connect(DB_PATH)
    df=add_price_features(load_prices(conn))
    sentiment=load_daily_sentiment(conn)
    df=pd.merge(df,sentiment,on=["symbol","date"],how="left")
    df["sentiment"]=df["sentiment"].fillna(0.0)
    df.to_sql("features",conn,if_exists="replace",index=False)
    conn.commit(); conn.close()
```

- `pd.merge(..., on=["symbol","date"], how="left")` = a **left join**: keep *every* price row, and attach sentiment where a matching (symbol, date) exists. Days with no news get `NaN`.
- `.fillna(0.0)` turns those `NaN`s into 0.0 = "neutral, no news." (Expected: with free-tier news covering only recent days, *most* of your ~100 price days legitimately have 0

sentiment.)

- `df.to_sql("features", conn, if_exists="replace", index=False)` writes the DataFrame straight into a `features` table. `if_exists="replace"` rebuilds it each run (safe — it's derived data). `index=False` = don't store pandas' row numbers as a column.

The final `print` shows the last 5 rows so you can eyeball that returns, SMAs, and sentiment look sane.

Part 7 — `backtest.py` (the finance-heavy file)

Job: turn features into a trading signal, simulate it honestly, print performance vs. buy-and-hold, and save the equity curve.

```python
COST=0.001
TRADING_DAYS=252
```

`COST` = 0.1% charged whenever you change position (Section E). `TRADING_DAYS` = ~252 market days/year, used to annualize the Sharpe ratio.

```python
def load_features(conn):
    df=pd.read_sql_query("SELECT * FROM features",conn)
    df["date"]=pd.to_datetime(df["date"])
    return df.sort_values(["symbol","date"]).reset_index(drop=True)
```

Load the `features` table, parse dates, sort by ticker+date (mandatory before any `shift`/return math).

```python
def build_signal(df):
    trend_up=df["sma_fast"]>df["sma_slow"]
    sentiment_ok=df["sentiment"]>=0
    df["signal"]=np.where(trend_up & sentiment_ok,1,0)
    return df
```

The strategy rule. `trend_up` is a True/False column: is the 5-day average above the 20-day (short-term momentum up)? `sentiment_ok`: is news not negative? `np.where(condition, 1,`

`0)` = "put 1 where the condition is True, else 0." So `signal=1` (want to be in the market) only when the trend is up **and** sentiment isn't negative. Deliberately simple and rules-based — easy to defend, easy to reason about.

```python
def run_backtest(df):
    df["position"]=df.groupby("symbol")["signal"].shift(1).fillna(0)
    df["prev_position"]=df.groupby("symbol")["position"].shift(1).fillna(0)
    df["position_change"]=(df["position"]-df["prev_position"]).abs()
    df["strategy_return"]=(df["position"]*df["daily_return"]-(COST*df["position
    return df
```

This is the heart, and the part interviewers grill:

- `position = signal.shift(1)` — **the anti-lookahead step.** You only act on a signal the *day after* you see it, so today's holding is yesterday's signal. Shifting inside `groupby("symbol")` stops one ticker's last day leaking into the next ticker's first day. `.fillna(0)` sets the first day (no prior signal) to "out of market."
- `prev_position` = position shifted one more day, so you can detect changes.
- `position_change = |position − prev_position|` — this is 1 on any day you flipped in or out, 0 when you held steady. `.abs()` = absolute value.
- `strategy_return = position × daily_return − COST × position_change` — you earn the day's return *only if you were holding* (`position=1`), and you subtract the trading fee *only on days you changed*. `.fillna(0)` cleans up the first row's `NaN`.

```python
def metrics(returns):
    returns=returns.fillna(0)
    if len(returns)==0 or returns.std()==0:
        sharpe=0.0
    else:
        sharpe=(returns.mean()/returns.std())*np.sqrt(TRADING_DAYS)
    equity=(1+returns).cumprod()
    max_dd=(equity/equity.cummax()-1).min()
    total_return=equit.iloc[-1]-1 if len(equity) else 0.0
    active=returns[returns!=0]
    hit_rate=(active>0).mean() if len (active) else 0.0
    return total_return, sharpe, max_dd, hit_rate
```

Computes the four scorecard numbers (Section E):

- **sharpe** = mean/std × √252. The `if` guards against dividing by zero (a flat strategy with no variation).
- **equity** = `(1+returns).cumprod()` — the account curve. Example: returns `[0.01, -0.02, 0.03]` → growth factors `[1.01, 0.98, 1.03]` → cumulative product `[1.01, 0.9898, 1.0195]`.
- **max_dd** = `(equity / equity.cummax() - 1).min()`. `equity.cummax()` is the running peak; dividing current equity by the peak gives how far below the peak you are; `.min()` finds the worst point. A negative number like −0.15 = a 15% drawdown.
- **total_return** = final equity − 1 (e.g., 1.08 → +8%).
- **hit_rate** = of days you actually traded (`returns != 0`), the share that were positive.

🐞 **Real bug here:** `total_return=equit.iloc[-1]-1` — `equit` is a typo; it should be `equity.iloc[-1]`. As written this crashes with `NameError: name 'equit' is not defined` the moment `metrics()` runs. **Fix:** change `equit` to `equity`. (Also a tiny cosmetic thing: `len (active)` and `len (returns)` have a stray space before the paren — legal Python, just unusual style.)

python

```
def main():
    conn=sqlite3.connect(DB_PATH)
    df=run_backtest(build_signal(load_features(conn)))
    port=df.groupby("date")["strategy_return"].mean().fillna(0)
    bench=df.groupby("date")["daily_return"].mean().fillna(0)
```

Load → signal → backtest in one chained line. Then:

- `port` = the **portfolio** return each day = average of the five tickers' strategy returns (equal-weight: you split money evenly across the five).
- `bench` = the **benchmark** = average daily return of just holding all five (buy-and-hold). This is what you must beat to have done anything useful.

python

```
    for sym,sub in df.groupby("symbol"):
        tr,sh,dd,hr=metrics(sub["strategy_return"])
        print(f"{sym}: return={tr:+.2%}, sharpe={sh:.2f}, maxDD={dd:.2%}, hit={
    s=metrics(port)
    b=metrics(bench)
```

Print per-ticker metrics (looping each ticker's slice), then compute portfolio (`s`) and benchmark (`b`) metrics. `:+.2%` formats a number as a percentage with a sign (0.083 → `+8.30%`); `:.2f` = two decimals; `:.0%` = whole-number percent. *(Minor inconsistency: the portfolio/benchmark `print`s format `maxdd` with `:.2f` instead of `:.2%`, so drawdown shows as e.g. `-0.15` rather than `-15%`. Not a bug, just reads differently from the per-ticker lines.)*

```python
    out=pd.DataFrame({
        "date":port.index,
        "strategy_equity":(1+port).cumprod().values,
        "benchmark_equity":(1+bench).cumprod().values
    })
    out.to_sql("backtest_equity",conn,if_exists="replace",index=False)
    conn.commit(); conn.close()
```

Builds a small table of both equity curves over time (`port.index` is the dates) and saves it as `backtest_equity` — this is what your dashboard will later plot. `.values` extracts the raw numbers from the pandas Series.

Part 8 — Bugs & fixes summary

Fix these before your next run:

1. **`backtest.py` — crash.** `total_return=equit.iloc[-1]-1` → typo. Change `equit` to `equity`. Without this, `metrics()` throws `NameError` immediately.
2. **`llm_extract.py` — latent crash.** `float(result.get("confidence"))` breaks if the model omits `confidence`. Change to `float(result.get("confidence", 0.0))`.
3. **`backtest.py` — cosmetic.** Portfolio/benchmark prints use `:.2f` for `maxdd`; use `:.2%` to match the per-ticker lines and show a real percentage.
4. **Awareness, not a bug — `fetch_news.py` drops its tables each run.** Don't re-run it after `llm_extract.py` unless you want fresh news (it can leave orphaned analysis rows).

Run order recap (data dependencies)

```
fetch_prices.py   → daily_prices
fetch_news.py     → news_articles, news_ticker_sentiment
llm_extract.py    → news_llm_analysis   (reads the two news tables)
build_features.py → features            (reads prices + news)
backtest.py       → backtest_equity     (reads features)
```

`read_prices.py` and `generate_signals.py` are standalone inspection scripts — run them anytime after their source tables exist; they don't feed the pipeline.

You built all of this, so you can absolutely defend it. If any single concept above still feels fuzzy — `groupby`/`transform`, the `shift(1)` lookahead fix, JOINs, or the Sharpe/drawdown math — that's the highest-value thing to drill next, because those are exactly what an interviewer will poke at.